Designing a Complex Family Tree App

https://gemini.google.com/app/92494a77cf689cdd

User prompt: I'd like to build a web application to display my family tree in a user-friendly way. I'd like to build the API in C#.NET and the front-end using a suitable tech. I'm familiar with React, Blazor, and JAMStack, specifically 11ty. My current site is built using 11ty, so if we can build it in something that will integrate with that, great. I'd like to add authentication, so that only users who I have allowed access to will be able to see the content. The front-end will need to display the tree as a whole, and allow users to zoom in and click on individuals in the tree to see more data about them. I'd also like to build a map integration, perhaps with Google Maps where each person's location will be plotted on a map and displayed along with their details. The family itself is extremely complicated, with lots of re-marriages and multiple children with unknown fathers in some cases. Please propose a design.

Response: Building a digital home for a complex family tree is a great project, but because your tree has "edge cases" like multiple marriages and unknown parentage, the **data schema** is actually more important than the UI.

Since your current site is on **11ty**, you have a choice: keep it static for speed or move to a more "app-like" framework. Given the zoomable tree and auth requirements, a **Hybrid approach** is best.

1. The Architecture

To keep your 11ty site but add these heavy features, I recommend a **"Sidecar App"** approach.

- **API:** C# .NET 8 Web API (hosted on Azure/AWS).
- **Database:** PostgreSQL with **EF Core**. (Relational DBs are better for complex lineage than NoSQL).
- **Front-end:** A **React** application embedded into an 11ty page or hosted on a subdomain (e.g., `family.yourdomain.com`).
    - *Why React over Blazor?* For complex, zoomable "Canvas" or "SVG" interactions (like a family tree), the JavaScript ecosystem (D3.js or React Flow) is significantly more mature.
- **Auth: Auth0** or **Clerk**. They handle "Invite Only" workflows beautifully without you having to build a login system from scratch in C#.

2. Handling the "Messy" Data (The Schema)

Standard "ParentID" columns in a User table will fail you. You need a **Graph-based Relational Schema**.

Instead of a simple hierarchy, use a **Relationship Table** to bridge individuals.

Person Table	Relationship Table
`Id`	`Id`
`FirstName`	`PersonA_Id` (The "Source")
`LastName`	`PersonB_Id` (The "Target")
`BirthDate`	`RelType` (Parent, Spouse, Sibling)
`Metadata` (JSONB)	`IsBiological` (Boolean)

> **Pro Tip:** For unknown fathers, use a "Placeholder" entity in the database or allow the `FatherId` to be null while linking the child to a "Union" entity rather than a specific couple.

3. The Front-End Experience

The Tree View (D3.js or React Flow)

Standard CSS layouts can't handle complex trees. Use **React Flow** or **D3.js**.

- **Zoom/Pan:** These libraries handle infinite canvases out of the box.
- **Complexity:** They can render "Non-Tree" graphs (where lines cross or loop back), which is vital for re-marriages.

The Map Integration

Since you have C# on the back-end, you can use the **NetTopologySuite** library to handle spatial data.

- **Frontend:** Use `react-google-maps`.

- **Feature:** When a user clicks a name in the tree, the map "flies" to their birthplace. When they click a pin on the map, a sidebar opens with that person's 11ty-generated profile.

4. Implementation Steps

Phase 1: The .NET API

1. Create a `Person` model and a `Relationship` model.
2. Implement **JWT Authentication** so the React app can talk to it securely.
3. Create an endpoint `GET /api/tree` that returns a flat list of nodes and edges (links).

Phase 2: The React Component

1. Build a simple React app.
2. Use `react-query` to fetch your C# data.
3. Integrate **React Flow** to draw the lines between your family members.

Phase 3: The 11ty Integration

Instead of rebuilding your whole site, treat the Family Tree as a **"Single Page App" (SPA).** 11ty will build your headers, footers, and static bios, while the React app handles the interactive "Map" and "Tree" pages.

Comparison of UI Libraries

Library	Best For	Learning Curve
D3.js	Custom, highly artistic trees	Steep
React Flow	Interactive, draggable nodes	Medium
GoJS	Very complex logical diagrams	Medium (Paid)

Would you like me to draft the C# Entity Framework models for that "Relationship" table to get you started?

User prompt: Let's have a look at your suggestions for the Entity Framework models.

Response: To handle the complexity of re-marriages, step-parents, and unknown fathers, we should move away from a "flat" hierarchy. Instead, we'll use a **"Union" (or Partnership) model**.

In this design, children aren't linked directly to parents; they are linked to a **Union**. This allows you to define a relationship between two people (married, unmarried, or "unknown partner") and then attach children to that specific pairing.

1. The Core Entities

Here is how you would structure the models in C# using Entity Framework Core:

```
public class Person
{
    public int Id { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public string? LastName { get; set; }
    public DateTime? BirthDate { get; set; }
    public string? BirthPlace { get; set; } // For your Google Maps integration
    public double? Latitude { get; set; }   // For the Map Pin
    public double? Longitude { get; set; }  // For the Map Pin

    // Navigation properties
    public ICollection<UnionMember> Unions { get; set; } = new List<UnionMember>();
    public int? ParentUnionId { get; set; }
    public Union? ParentUnion { get; set; } // The union this person was born into
}

public class Union
{
```

```
    public int Id { get; set; }
    public DateTime? MarriageDate { get; set; }
    public string UnionType { get; set; } = "Marriage"; // e.g., Marriage, DeFacto, Unknown

    // Navigation properties
    public ICollection<UnionMember> Partners { get; set; } = new List<UnionMember>();
    public ICollection<Person> Children { get; set; } = new List<Person>();
}

public class UnionMember
{
    public int PersonId { get; set; }
    public Person Person { get; set; } = null!;

    public int UnionId { get; set; }
    public Union Union { get; set; } = null!;
}
```

2. Why this solves your "Complications"

- **Multiple Marriages:** A `Person` can be part of many `Unions`. EF Core will track each one via the `UnionMember` join table.
- **Unknown Fathers:** If the father is unknown, you create a `Union` with only one `UnionMember` (the mother). You can still attach the child to that Union.
- **Re-marriages with Children:** Since children are linked to a `UnionID` and not a `PersonID`, it is crystal clear which "set" of parents a child belongs to, even if the mother has five children by three different partners.

3. The "Map" Data Strategy

Since you mentioned Google Maps, I added `Latitude` and `Longitude` to the `Person` model. In your .NET API, you can create a specific "Map Data" DTO (Data Transfer Object) to keep the payload light:

```
public record MapPointDto(
    int Id,
    string FullName,
    double Lat,
    double Lng,
    string EventType // e.g., "Birth", "Death", "Residence"
);
```

4. Fluent API Configuration

Because these relationships can be "circular" (a person belongs to a union, and a union contains persons), you'll need to tell EF Core how to handle the foreign keys in your `DbContext`:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Define Many-to-Many for Partnerships
    modelBuilder.Entity<UnionMember>()
        .HasKey(um => new { um.PersonId, um.UnionId });

    // Link Children to their Union of origin
    modelBuilder.Entity<Person>()
        .HasOne(p => p.ParentUnion)
        .WithMany(u => u.Children)
        .HasForeignKey(p => p.ParentUnionId)
        .OnDelete(DeleteBehavior.Restrict);
}
```

Next Steps

This schema is robust enough to handle a "messy" family tree without breaking.

Would you like me to show you how the `GET /api/tree` controller logic would look to transform this data into a JSON format that a library like React Flow can understand?

User prompt: Please summarise this proposal for NotebookLM.

Response: This summary is structured for **NotebookLM**, focusing on the technical architecture, data integrity for complex lineages, and the integration strategy for your existing 11ty site.

Project Overview: Hybrid Family Tree Web App

The goal is to build a high-performance, secure family tree application capable of handling complex genealogical "edge cases" (re-marriages, unknown parentage, and non-traditional structures). The solution integrates a modern **C# .NET API** with a **React-based interactive front-end**, embedded within an existing **11ty** static site.

1. Core Architecture (The "Sidecar" Model)

- **Backend:** C# .NET 8 Web API. Provides a structured RESTful interface for data retrieval and authentication.
- **Database:** PostgreSQL (Relational). Chosen over NoSQL to strictly manage complex familial links via Entity Framework (EF) Core.
- **Frontend:** A React Single Page App (SPA). React is selected over Blazor for its superior ecosystem of canvas-based libraries (D3.js, React Flow) required for "infinite" zoomable trees.
- **Hosting:** The React app functions as a "sidecar" to the 11ty site, likely hosted on a subdomain or a specific route (e.g., `/tree`).

2. Data Modeling for Complexity

To solve for "messy" family structures, the design moves away from simple "ParentID" fields to a **Graph-based Union Schema**:

- **The Union Entity:** Instead of linking children to individuals, children are linked to a "Union" (a partnership).
- **Handling Edge Cases:** * **Re-marriages:** A person can be a member of multiple Union entities.
    - **Unknown Fathers:** A Union can exist with only one known member (the mother) while still acting as the anchor point for children.
- **Spatial Data:** Each `Person` entity includes `Latitude` and `Longitude` fields for Google Maps integration.

3. Key Features & Tools

- **Interactive Visualization:** Utilizes **React Flow** or **D3.js** for a zoomable, pannable SVG/Canvas interface. These libraries handle "non-tree" graphs (where lineages cross or loop) better than standard HTML.
- **Geospatial Mapping:** Integration with **Google Maps API** (via `react-google-maps`). Features include "Fly-to" animations where clicking a person in the tree focuses the map on their birth location.
- **Security:** **Auth0** or **Clerk** for authentication. This allows for an "Invite-Only" access model, ensuring sensitive family data is not public.

4. Technical Stack Summary

Layer	Technology
Language/Framework	C# .NET 8
ORM	Entity Framework Core
Frontend Library	React (Vite-based)
Static Site Generator	11ty (Existing)
Tree Rendering	React Flow / D3.js
Authentication	Auth0 (OIDC/OAuth2)

Would you like me to generate a sample JSON payload that the .NET API would send to the React front-end to render these complex unions?

User prompt: Please create a specification I can give to Claude Code to build this application. Include sample JSON payloads.

Response: Here is a complete, production-ready project specification designed specifically for **Claude Code** (or any agentic developer tool). You can save this file as `SPEC.md` or pass it directly to Claude Code to initiate construction.

Application Specification: Family Tree "Sidecar" Web App

1. Executive Summary & Goals

Build an invite-only, interactive family tree and geospatial mapping application capable of handling complex non-traditional genealogical lineages (multiple marriages, step-children, unknown parentage).

The solution will consist of:

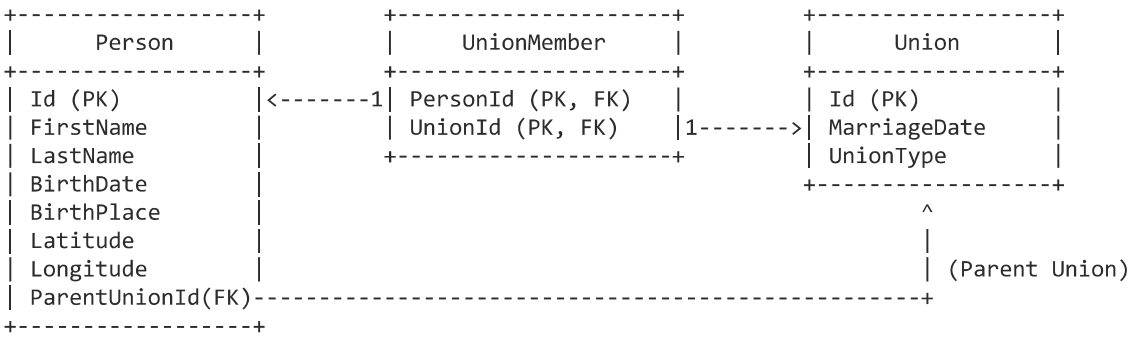
1. **Backend:** C# .NET 8 Web API managing a PostgreSQL relational database.
2. **Frontend App:** React (Vite) Single Page Application rendering an infinite zoomable canvas and interactive Google Map.
3. **Static Integration:** The React SPA will embed directly into an existing **11ty (Eleventy)** static site or run on a dedicated path/subdomain.
4. **Auth:** Invite-only authentication powered by Auth0 or Clerk issuing JWTs.

2. Technical Stack & Dependencies

- **Backend:** C# / .NET 8.0 Web API, Entity Framework Core 8, Npgsql (PostgreSQL), System.IdentityModel.Tokens.Jwt.
- **Frontend:** React 18, Vite, React Flow (`@xyflow/react`), `@vis.gl/react-google-maps`, `@tanstack/react-query`, `axios`, `lucide-react`.
- **Database:** PostgreSQL 16+.
- **Integration Layer:** 11ty shortcode / iframe / SPA embedding script.

3. Data Architecture (Union-Based Graph Schema)

To handle complex parentage without flat hierarchical limits, children are attached to a **Union** (relationship/partnership) rather than directly to a parent ID.



EF Core Entity Definitions

```
namespace FamilyTree.Api.Models;

public class Person
{
    public int Id { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public string? LastName { get; set; }
    public DateTime? BirthDate { get; set; }
    public string? BirthPlace { get; set; }
    public double? Latitude { get; set; }
    public double? Longitude { get; set; }
    public string? Notes { get; set; }

    // Relationships
    public int? ParentUnionId { get; set; }
    public Union? ParentUnion { get; set; } // The union/partnership this person was born into

    public ICollection<UnionMember> Unions { get; set; } = new List<UnionMember>();
}

public class Union
```

```
{
    public int Id { get; set; }
    public DateTime? MarriageDate { get; set; }
    public string UnionType { get; set; } = "Partnership"; // e.g. "Marriage", "DeFacto", "SingleParent"

    public ICollection<UnionMember> Partners { get; set; } = new List<UnionMember>();
    public ICollection<Person> Children { get; set; } = new List<Person>();
}

public class UnionMember
{
    public int PersonId { get; set; }
    public Person Person { get; set; } = null!;

    public int UnionId { get; set; }
    public Union Union { get; set; } = null!;
}
```

4. API Endpoints & Sample JSON Payloads

All endpoints (except Auth validation) require a valid JWT header: `Authorization: Bearer <TOKEN>`.

Endpoint 1: `GET /api/v1/tree`

Returns nodes and edges structured specifically for rendering in canvas graph tools like **React Flow**.

Sample Response Payload:


```
{
  "nodes": [
    {
      "id": "p-1",
      "type": "personNode",
      "data": {
        "id": 1,
        "fullName": "Jane Doe",
        "birthDate": "1955-04-12",
        "birthPlace": "Nottingham, UK",
        "lat": 52.9548,
        "lng": -1.1581
      }
    },
    {
      "id": "p-2",
      "type": "personNode",
      "data": {
        "id": 2,
        "fullName": "John Smith",
        "birthDate": "1952-09-20",
        "birthPlace": "London, UK",
        "lat": 51.5074,
        "lng": -0.1278
      }
    },
    {
      "id": "u-101",
      "type": "unionNode",
      "data": {
        "unionId": 101,
        "unionType": "Marriage",
        "marriageDate": "1978-06-15"
      }
    },
    {
      "id": "p-3",
      "type": "personNode",
      "data": {
        "id": 3,
        "fullName": "Alice Smith",
        "birthDate": "1982-11-03",
```

```
        "birthPlace": "Manchester, UK",
        "lat": 53.4808,
        "lng": -2.2426
      }
    }
  ],
  "edges": [
    { "id": "e-p1-u101", "source": "p-1", "target": "u-101", "type": "partner" },
    { "id": "e-p2-u101", "source": "p-2", "target": "u-101", "type": "partner" },
    { "id": "e-u101-p3", "source": "u-101", "target": "p-3", "type": "child" }
  ]
}
```

Endpoint 2: `GET /api/v1/map/locations`

Provides spatial data optimized for rendering marker pins on Google Maps.

Sample Response Payload:

```
{
  "locations": [
    {
      "personId": 1,
      "fullName": "Jane Doe",
      "eventType": "Birth",
      "locationName": "Nottingham, UK",
      "latitude": 52.9548,
      "longitude": -1.1581
    },
    {
      "personId": 2,
      "fullName": "John Smith",
      "eventType": "Birth",
      "locationName": "London, UK",
      "latitude": 51.5074,
      "longitude": -0.1278
    }
  ]
}
```

Endpoint 3: `POST /api/v1/persons`

Adds a new person and optionally connects them to an existing union or registers a new union.

Sample Request Payload:

```
{
  "firstName": "Arthur",
  "lastName": "Pendelton",
  "birthDate": "1988-02-14",
  "birthPlace": "Edinburgh, UK",
  "latitude": 55.9533,
  "longitude": -3.1883,
  "parentUnionId": 101,
  "notes": "Biological child from marriage 1"
}
```

5. Frontend & Visualization Requirements

1. **Tree Canvas (React Flow):**
    - Custom Node Component `personNode`: Renders avatar/initials, full name, and birth date. Clicking a node opens a side panel with full person details.
    - Custom Node Component `unionNode`: Small connector node representing a partnership/marriage. Lines draw from parents -> `unionNode` -> children.
    - Supports pan, pinch-to-zoom, and mini-map navigation.
2. **Map Drawer/Tab (Google Maps):**

- Uses `@vis.gl/react-google-maps`.
- Displays map markers for all family locations.
- **Bidirectional Interaction:**
  - Clicking a marker highlights the person in the Tree view.
  - Clicking "View on Map" in a person's detail sidebar triggers a `map.panTo()` / "Fly-To" animation to their birth location.

3. **11ty Integration Strategy:**
   - Configure Vite to build single-bundle output assets (`/dist/family-app.js` and `/dist/family-app.css`).
   - In 11ty, create a page template `family-tree.njk` containing a mounting target `<div id="family-tree-app"></div>` and load the compiled bundle.

6. Implementation Checklist for Claude Code

When starting this build, follow these execution steps sequentially:

1. **Backend Initialization:**
   - Create solution `FamilyTreeApp.sln` and Web API project `FamilyTree.Api`.
   - Set up EF Core with PostgreSQL connection strings.
   - Implement entities (`Person`, `Union`, `UnionMember`), `DbContext`, and EF migrations.
   - Write JWT validation middleware configured for Auth0/Clerk issuer.
   - Implement `TreeController` and `MapController` returning structured graph data.
2. **Frontend Initialization:**
   - Bootstrap Vite React TypeScript project inside `frontend/`.
   - Install React Flow, Google Maps wrapper, and TanStack Query.
   - Implement custom `PersonNode` and `UnionNode` components for React Flow.
   - Create map view syncing selection states with tree view.
3. **11ty Integration Pipeline:**
   - Configure `vite.config.ts` build commands to output assets directly into the 11ty static output directory (`_site/assets/`).
